

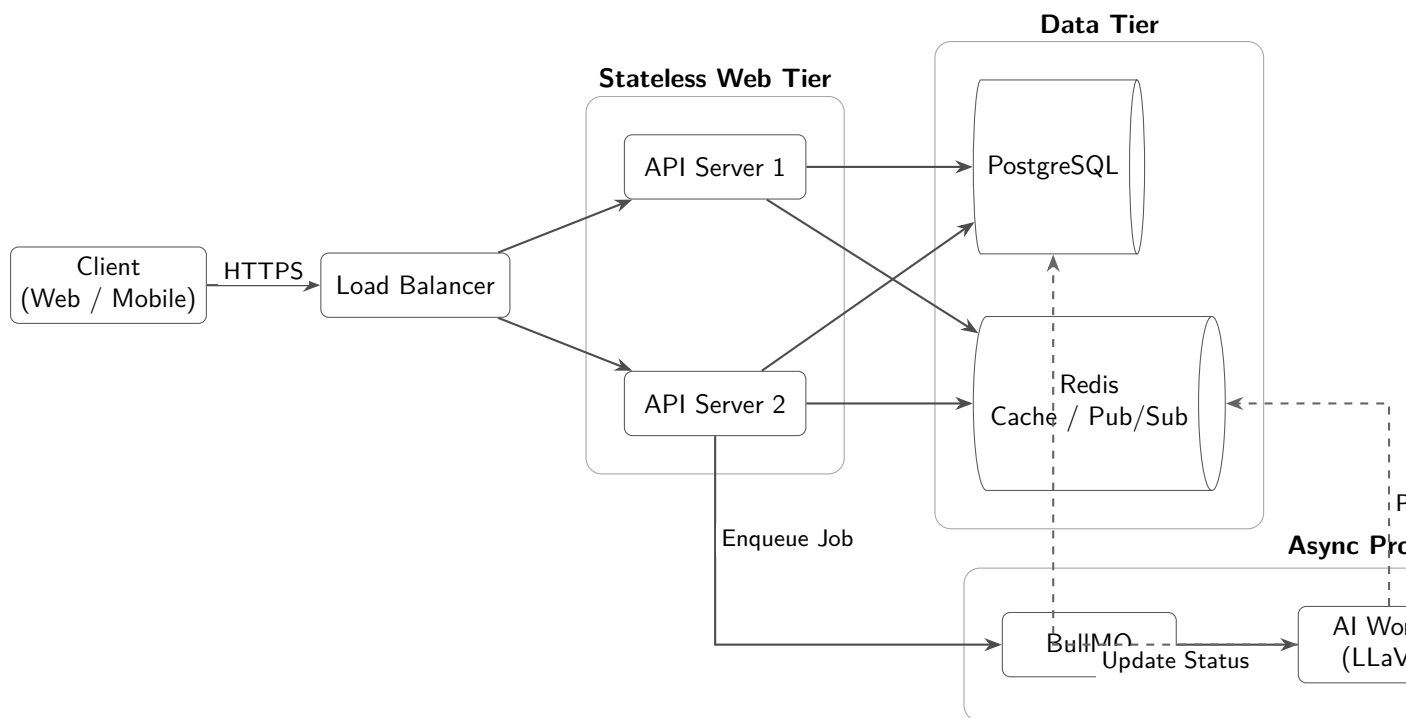
Macrolytics

Backend System Architecture & API Design

System Design Documentation

1 System Overview

Macrolytics relies on a decoupled, distributed backend architecture designed to handle deterministic HTTP requests, asynchronous AI inference, and atomic data aggregation concurrently. The system prioritizes a completely stateless web tier, event-driven background processing, and strict multi-agent routing to ensure high availability and low latency.



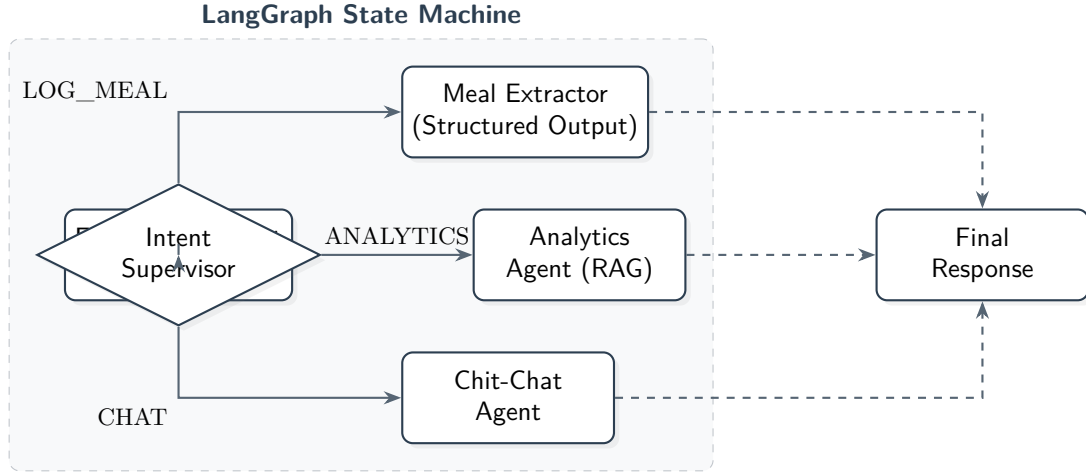
2 Architectural Decisions & Complexities

The backend relies on several specific design choices to optimize for scalability, cost, and the unique constraints of local Large Language Models (LLMs).

- **Stateless Web Tier via Encrypted Chat Context (Innovation):** Traditional chat applications utilize stateful servers (binding a user to a specific server via WebSockets/sticky sessions) to maintain conversational memory. To keep our web tier completely stateless and horizontally scalable, we implemented a cryptographic context-passing innovation.
 - The AI agent’s conversation summary and recent message array are encrypted using A256GCM on the server and sent to the client.
 - The client stores this token and passes it back on the next request.
 - *Justification:* This forces the LLM context to live client-side securely, completely eliminating server-side memory bottlenecks and allowing any API server to seamlessly pick up the conversation.
- **Redis for Caching and Pub/Sub Messaging:**
 - **Caching:** Daily macro aggregates are stored using Redis Hashes. Operations utilize `hIncrByFloat` for thread-safe, atomic increments, providing $O(1)$ read complexity for the dashboard, drastically reducing relational database load.
 - **Pub/Sub Model:** We utilize Redis Pub/Sub to solve the asynchronous communication gap between the background AI workers and the stateless API servers. When a BullMQ worker finishes processing a vision task, it publishes a completion event to Redis. Subscribed API servers instantly receive this event and can push real-time Server-Sent Events (SSE) to the client, entirely avoiding inefficient database polling.
- **Asynchronous Processing for Vision AI (BullMQ):**
 - Local vision model inference (LLaVA) takes 10–30 seconds. Synchronous execution would block the Node.js event loop and trigger HTTP timeouts.
 - *Justification:* Event-driven decoupling. The API instantly returns a HTTP 202 (Accepted) with a `jobId`, while BullMQ manages retries, failure states, and concurrency limits for the GPU-bound workers safely in the background.
- **Secure S3-to-Local Data Pipeline:**
 - Local LLMs lack the native architecture to securely resolve remote AWS S3 URIs.
 - *Justification:* The worker acts as a secure proxy. It downloads the S3 image to local `/temp`, converts it to Base64 for the LLM, and immediately deletes both the S3 object and local file upon completion. This strict ephemeral storage strategy guarantees user privacy and minimizes cloud storage costs.
- **Relational Database (PostgreSQL):**
 - *Justification:* Nutrition logging requires complex, paginated time-range queries and aggregate mathematical functions (`SUM`). NoSQL data stores handle long-tail random access poorly when complex relations are required. PostgreSQL natively supports the ACID transactions required for accurate macro tracking.

3 AI Agent Orchestration (LangGraph)

A standard ReAct (Reasoning and Acting) loop fails frequently on smaller local models (e.g., Llama 3.1 8B) due to high cognitive load, resulting in infinite tool-calling loops and dropped parameters. To mitigate this, we implemented a **Supervisor Orchestrator Pattern**.



3.1 Orchestration Logic

- **Intent Routing:** The Supervisor node utilizes `withStructuredOutput` to deterministically classify the user's intent. The LLM does not execute tools; it purely routes.
- **Domain Isolation:** By routing to specialized sub-agents, the system drastically shrinks the prompt size and cognitive load for each task. The *Meal Extractor* forces JSON generation bypassing conversational text entirely, guaranteeing correct database insertion.
- **Psychological Circuit Breakers:** To prevent LangGraph recursion limits (ping-ponging), the Supervisor prompt contains dynamic stop-conditions: *"If the last message in history is an AI confirmation, output FINISH."*

4 API Endpoints

The backend exposes a RESTful API, completely decoupled from the frontend, supporting JSON payloads and Bearer Token authorization.

4.1 Core Nutrition & Aggregation

All listing APIs support standard pagination (**limit**, **page**) and specific time-range query parameters.

Method	Endpoint	Description
GET	/nutrition/entries	Retrieves paginated food entries. Filterable via startDate , endDate , and mealType .
POST	/nutrition/entries	Creates a new food entry, executes SQL insert, and increments the daily Redis Hash atomically.
DELETE	/nutrition/entries/:id	Hard deletes an entry and recalculates the daily Redis cache.
GET	/nutrition/reports/weekly	Returns a 7-day chronological aggregation of Calories, Protein, Carbs, and Fat for chart rendering.

4.2 AI & Asynchronous Jobs

Method	Endpoint	Description
POST	/uploads/presigned-url	Generates a secure, temporary AWS S3 pre-signed URL for direct client-to-cloud image uploads, saving API bandwidth.
GET	/jobs/:jobId	Polls Redis for the execution status of a BullMQ vision task (Pending, Completed, Failed).
POST	/agent/chat	Accepts the encryptedContext token and user prompt. Streams real-time Server-Sent Events (SSE) back to the client.